Flux 14 января 2011 в 16:49

Trie, или нагруженное дерево

Алгоритмы*

Из песочницы

Здравствуй, Хабрахабр. Сегодня я хочу рассказать о такой замечательной структуре данных как **словарь на нагруженном дереве**, известной также как **префиксное дерево**, или **trie**.

Что это ?

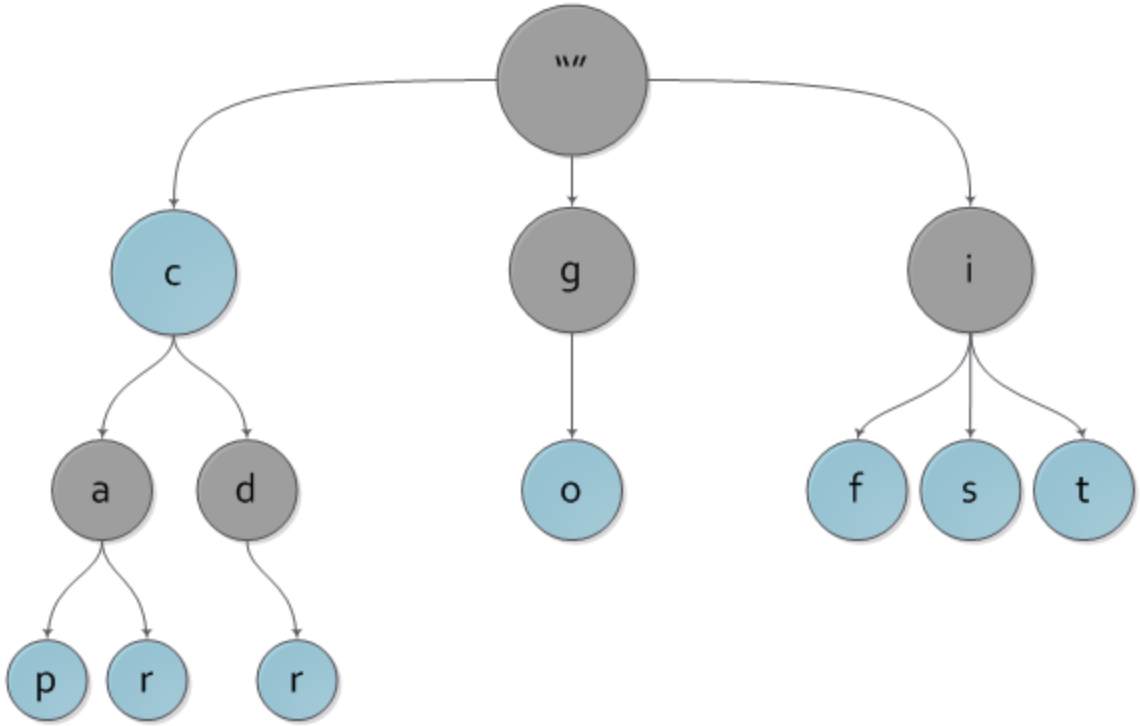
Нагруженное дерево — структура данных реализующая интерфейс ассоциативного массива, то есть позволяющая хранить пары «ключ-значение». Сразу следует оговорится, что в большинстве случаев ключами выступают строки, однако в качестве ключей можно использовать любые типы данных, представимые как последовательность байт (то есть вообще любые).

Как это работает ?

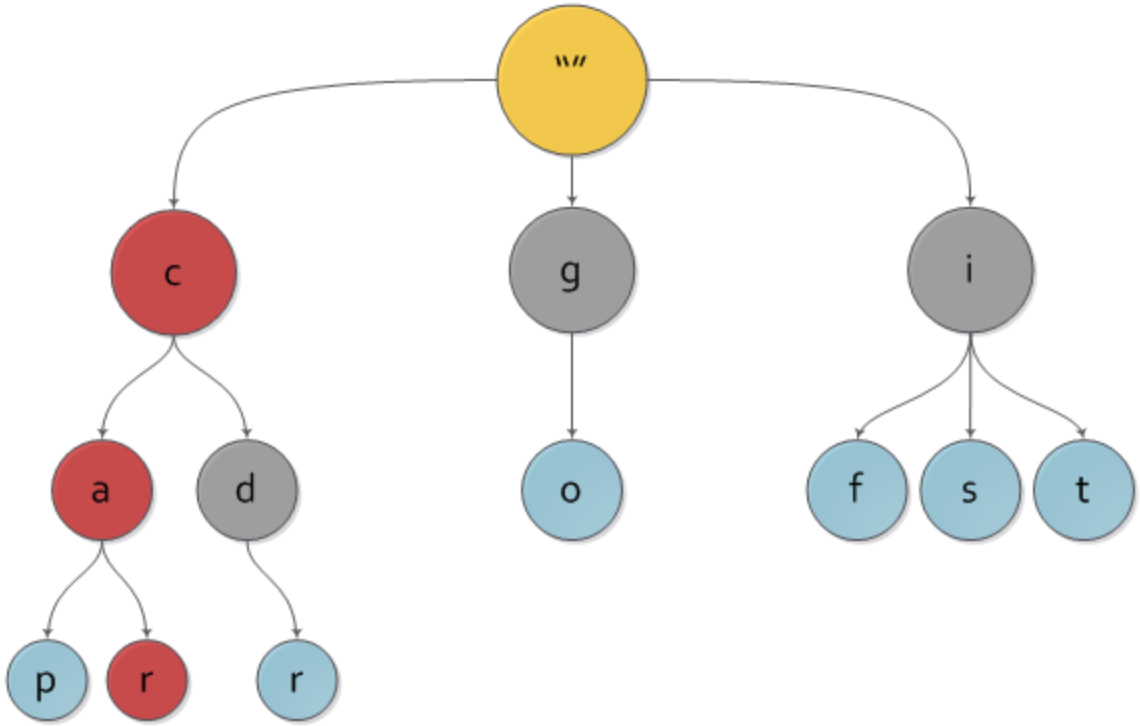
Нагруженное дерево отличается от обычных n-арных деревьев тем, что в его узлах не хранятся ключи. Вместо них в узлах хранятся односимвольные метки, а ключем, который соответствует некоему узлу является путь от корня дерева до этого узла, а точнее строка составленная из меток узлов, повстречавшихся на этом пути. В таком случае корень дерева, очевидно, соответствует пустому ключу.

На рисунке вы можете наблюдать пример нагруженного дерева с ключами *c, cap, car, cdr, go, if, is, it*.

+68 85K 318



И то же самое дерево с выделенным на нем ключем *car*.



Сразу видно, что наше дерево содержит «лишние» ключи, ведь любому узлу дерева соответствует единственный путь до него от корня, а значит и некоторый ключ. Чтобы избежать проблемы с «лишними» ключами, каждому узлу дерева добавляется булева характеристика, указывающая, является ли узел реально существующим либо промежуточным по дороге в какой-либо другой.

Основные операции

Так как нагруженное дерево реализует интерфейс ассоциативного массива, в нем можно выделить три основные операции, а именно вставку, удаление и поиск ключа. Как и многие деревья, нагруженное дерево обладает свойством самоподобия, то есть любое его поддерево также является полноценным нагруженным деревом. Легко заметить, что все ключи в таком поддереве имеют общий префикс, (откуда и пошло название «префиксное дерево») а значит можно выделить специфичную для этого дерева операцию — получение всех ключей дерева с заданным префиксом за время O(|Prefix|).

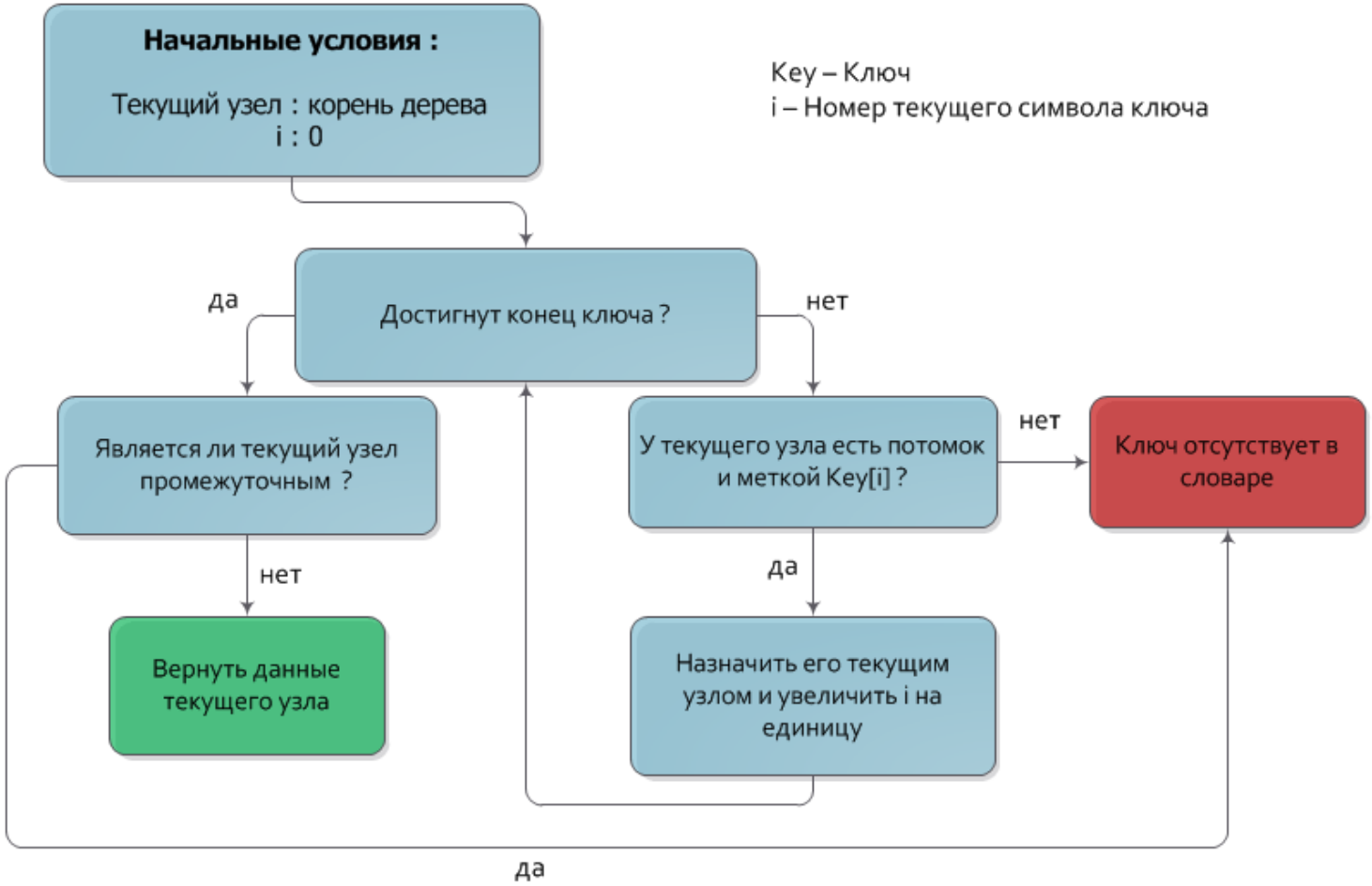
Поиск ключа

Как уже было сказано, ключ, соответствующий узлу — конкатенация меток узлов, содержащихся в пути от корня к данному узлу. Из этого свойства естественным образом следует алгоритм поиска ключа (как, впрочем, и алгоритмы добавления и удаления).

Пусть дан ключ Key, который необходимо найти в дереве. Будем спускаться из корня дерева на нижние уровни, каждый раз переходя в узел, чья метка совпадает с очередным символом ключа. После того как обработаны все символы ключа, узел, в котором остановился спуск и будет искомым узлом. Если в процессе спуска не нашлось узла с меткой, соответствующей очередному символу ключа, или спуск остановился на промежуточной вершине (вершине, не имеющей значения), то искомый ключ отсутствует в дереве.

Временная сложность этого алгоритма, очевидно, равна O(|Key|).

Более подробно алгоритм показан на блок-схеме:



Вставка

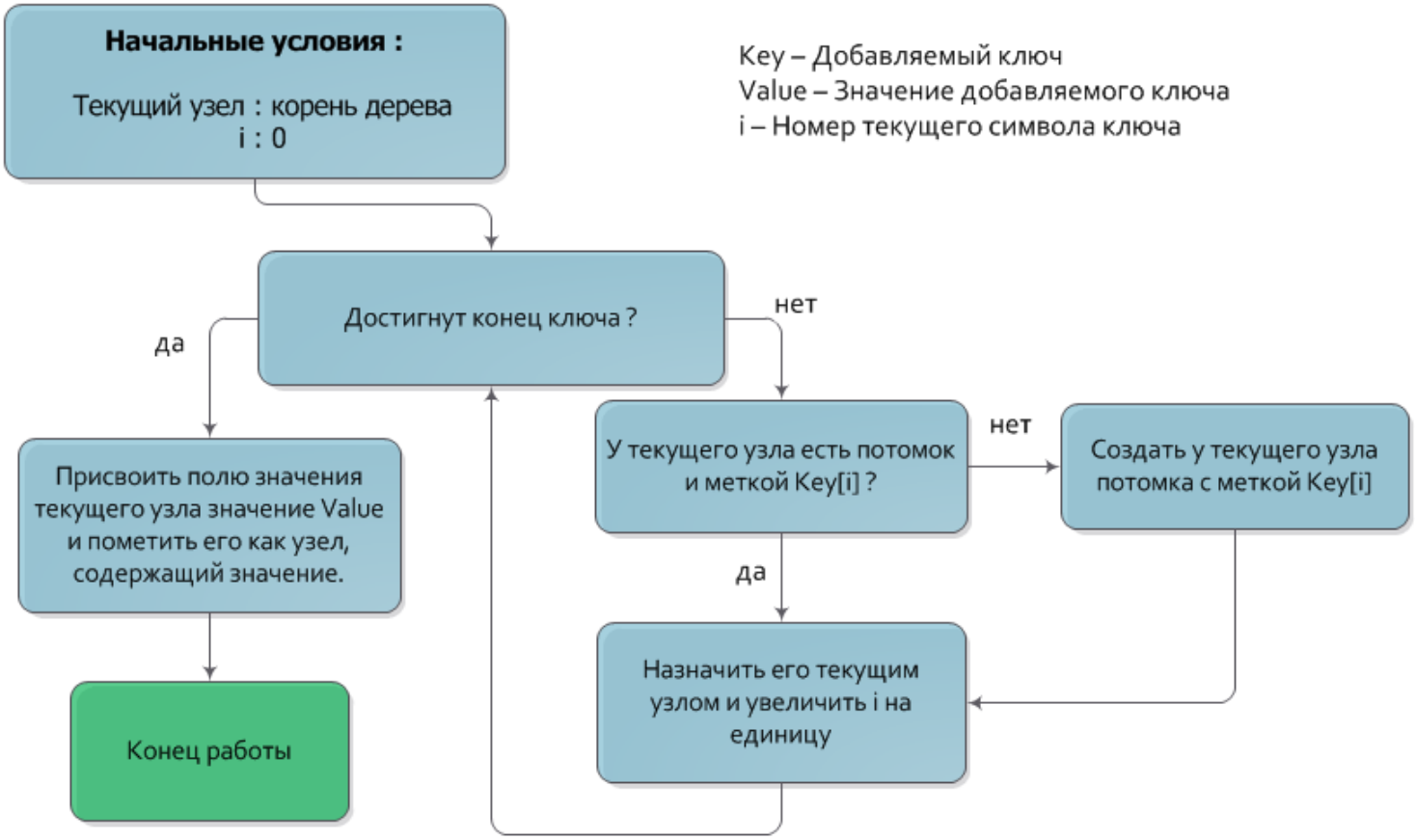
Алгоритм добавления ключа в дерево очень похож на алгоритм поиска.

Пусть дана пара из ключа Key и значения Value, которую нужно добавить. Как и в алгоритме поиска ключа, будем спускаться из корня дерева на нижние уровни, каждый раз переходя в узел, чья метка совпадает с очередным символом ключа. После того как обработаны все символы ключа, узел, в котором остановился спуск и будет узлом, которому должно быть присвоено значение Value (также, разумеется, узел должен быть помечен как имеющий значение). Если в процессе спуска отсутствует узел с меткой, соответствующей очередному символу ключа, то следует создать новый промежуточный узел с нужной меткой и назначить его потомком текущего

следует создать новый промежуточный узел с нужной меткой и назначить его потомком текущего.

Временная сложность добавления ключа — O(|Key|).

Иллюстрация алгоритма на блок-схеме:



Удаление

Удаление ключа также реализуется очень легко.

Пусть дан ключ Key, который необходимо удалить из дерева. Проведем поиск этого ключа. Если ключ существует в словаре, то зная узел, которому он соответствует, можно просто пометить его как промежуточный, сделав его «невидимым» для последующих поисков.
После этого можно подняться от «отключенного» узла к корню, попутно удаляя все узлы которые

являются листьями, однако экономия памяти в данном случае не существенна, а для

эффективного определения того, является ли узел листом потребуется вводить дополнительную характеристику узла.

Временная оценка алгоритма удаления — знакомое уже O(|Key|).

Требовательность к ресурсам

Нагруженное дерево по показателям потребления памяти/процессорного времени не уступает хэш-таблицам и сбалансированным деревьям, а иногда и превосходит их по этим параметрам.

Процессорное время

Сложность операций вставки, удаления и поиска — O(|Key|). Хотя сбалансированные деревья и выполняют эти операции за O(ln(n)) но в этой асимптотике скрыто время, необходимое для сравнения ключей, которое, в общем случае, составляет O(|Key|). С хэш-таблицами ситуация аналогична — хоть время доступа и составляет O(1+a), но взятие хэша (если он не предвычислен заранее, разумеется) занимает O(|Key|).

Графики со сравнением производительности нагруженных деревьев, хэш-таблиц и красно-черных деревьев можно посмотреть в английской википедии.

Память

По потреблению памяти нагруженное дерево часто выигрывает у хэш-таблиц и сбалансированных деревьев. Это связано с тем что у множества ключей в нагруженном дереве совпадают префиксы, и вместе с ними память которую они используют. Также, в отличии от сбалансированных деревьев, в нагруженном дереве нет необходимости хранить ключ в каждом узле.

Оптимизации

Существует 2 основных типа оптимизации нагруженного дерева:

- **Сжатие**. Сжатое нагруженное дерево получается из обычного удалением промежуточных узлов, которые ведут к единственному не промежуточному узлу. Например, цепочка промежуточных узлов с метками *a, b, c* заменяется на один узел с меткой *abc*.
- **Patricia**. Patricia нагруженное дерево получается из сжатого (или обычного) удалением промежуточных узлов, которые имеют одного ребенка.

Зачем все это нужно?

Собственно, область применения нагруженных деревьев огромна — их можно применять везде где требуется реализация интерфейса ассоциативного массива. Особенно нагруженные деревья удобны для реализации словарей, спеллчекеров и прочих T9 — то есть в задачах, где необходимо быстро получать наборы ключей с заданным префиксом. Также нагруженное дерево использует в своей работе небезизвестный алгоритм Ахо — Корасик.

Вот собственно и все. Надеюсь, читателю было интересно узнать об этой замечательной структуре данных, незаслуженно редко упоминаемой на Хабре.

Теги: Trie, нагруженное дерево, префиксное дерево, никто не читает теги

Хабы: Алгоритмы

Редакторский дайджест
Присылаем лучшие статьи раз в месяц

Электропочта

30 Карма 1.8 Рейтинг

@Flux

ML engineer

Реклама

Комментарии 29

ПОХОЖИЕ ПУБЛИКАЦИИ

1 февраля 2021 в 12:56

Дальше не читал

+47 9.9K 38 15 +15

4 июля 2018 в 18:30

Низкоуровневая реализация префиксного дерева trie на PHP

+22 5.3K 42 26 +26

13 марта 2018 в 10:19

Решаем задачи без самобалансирующихся деревьев в Python

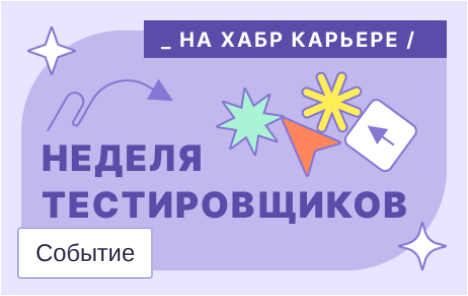
+21 11K 89 0

МИНУТОЧКУ ВНИМАНИЯ

Разместить

Анкета: попробуй себя в роли автора контент-студии Хабра

Пошаговый рецепт статьи о своём коде



Неделя тестировщиков на Хабр Карьере

ВАКАНСИИ

Frontend developer (JS React)
от 230 000 до 300 000 ₽ · SberTech · Москва

Эксперт курса «Инженер по тестированию» (не преподаватель)
до 40 000 ₽ · Алгоритмика · Можно удаленно

Product Owner
до 200 000 ₽ · BR Lab · Москва · Можно удаленно

Web Architect
до 200 000 ₽ · Animaccord · Можно удаленно

Инженер по прототипированию (FPGA -> ASIC)
от 180 000 ₽ · YADRO · Москва

Больше вакансий на Хабр Карьере

ЛУЧШИЕ ПУБЛИКАЦИИ ЗА СУТКИ

сегодня в 11:01

Апериодическая решётка Яндекс Станции 2: история, 3D-модели и акустические эксперименты

+45 3.2K 12 19 +19

сегодня в 10:04

Привлекательные структуры данных

+30 3K 37 7 +7

сегодня в 12:00

Новая версия Хабра. Ещё не всё потеряно, ещё не всё?... (часть первая)

+22 1.6K 11 14 +14

вчера в 16:00

Параллельные оболочки с xargs: Используем все процессорные ядра в UNIX и Windows

+22 2.6K 29 1 +1

сегодня в 10:01

Зуб мудрости, который перевернулся и растёт внутрь черепа

+21 3K 16 14 +14

Как устроен МКБ Реактор: история одного стартапа

Мегапост

Реклама

ЧИТАЮТ СЕЙЧАС

«Великое увольнение» продолжается: теперь с работы уходят даже боссы

11K 46 +46

Изобретатель из США собрал рабочий «математический велосипед»

6.7K 29 +29

web3 не будет: расходимся по домам

2K 2 +2

Эксперт по IT и телекоммуникациям рассказал о последствиях ухода из РФ Cisco и Microsoft

31K 130 +130

Росавиация отказалась подтверждать квалификацию пилотов, которые ищут работу за рубежом

7.1K 22 +22

Как мы в СИБУРе делаем дашборды для людей

Турбо

Реклама

© 2006–2022, Habr